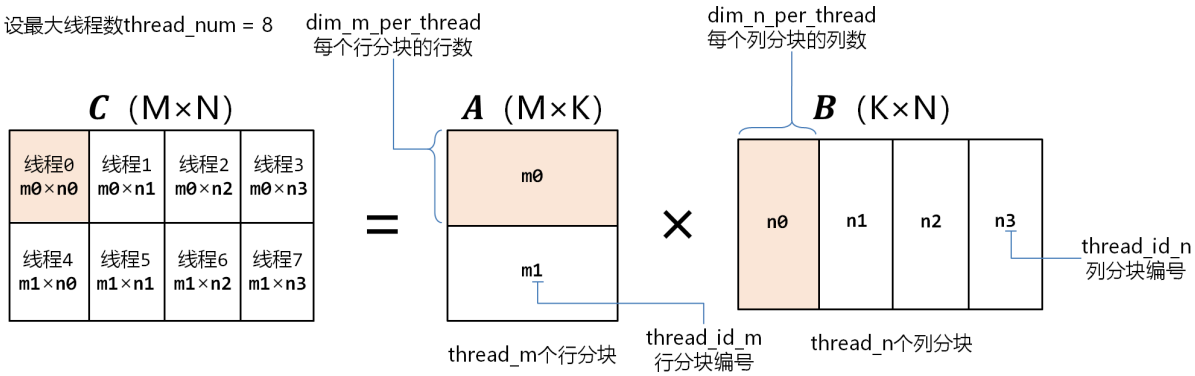
练习3 (选做)：基于OpenMP库和练习2实现的高性能计算内核实现任意形状的矩阵乘法计算

- **任务**：利用OpenMP库将练习2实现的单线程高性能矩阵乘法内核扩展到多核以实现计算更大规模的矩阵乘法的目标。
- **子任务**：
  - 矩阵乘法能够利用多线程实现对 , , 形状的计算；
  - 切分后不满足练习2内核要求大小的数据块允许采用Padding或另行设计内核的方式进行计算；
  - 自行设计各个维度的并行策略但线程总数不能超过函数参数中指定的线程总数；
  - 计算性能需超过基线。
- **目标**：学会利用不同层级的并行策略对程序进行加速。

> **练习说明**
>
> 本练习的并行优化策略属于开放性内容，没有严格意义上的最优方案。

0. 模板代码 `src/lab3/openmp_gemm_baseline.cpp` 的实现思路

模板代码 `src/lab3/openmp_gemm_baseline.cpp` 采用如下图所示的多线程矩阵分块算法：



第6行~第13行的 `get_parallel_thread_num` 函数使用最大线程数 `max_threads` 计算矩阵 `A` 的行分块数 `m_thread` 和矩阵 `B` 的列分块数 `n_thread` 。其中，矩阵 `A` 最多只被切分成2个行分块。

Loading [Contrib]/a11y/accessibility-menu.js

第15行`openmp_gemm_baseline`函数的`thread_num`参数表示计算使用的最大线程数，可通过命令行参数传递。

第22行~第26行使用编译指令描述线程创建的线程数、共享变量（矩阵`C`）、私有变量等配置信息。其后包含在大括号里的第27行~第100行是每个线程需要执行的代码。

第29行获取线程 ID。第一个线程 ID 为0，第二个线程 ID 为1，依此类推。

第31行、第32行将线程 ID 映射到矩阵`A`行分块的编号`thread_id_m`、矩阵`B`列分块的编号`thread_id_n`。例如，假设 ID 为6的线程映射到的行分块编号是1，列分块编号是2，则表示6号线程将计算矩阵`A`第1个行分块和矩阵`B`第2个列分块的矩阵乘法。

第36行、第41行分别计算矩阵`A`每个行分块的行数`dim_m_per_thread`、矩阵`B`每个列分块的列数`dim_n_per_thread`。

第48行~第52行计算矩阵`A`第`thread_id_m`个行分块的开始行号`thread_m_start`和结束行号`thread_m_end`。

第55行~第59行计算矩阵`B`第`thread_id_n`个列分块的开始列号`thread_n_start`和结束列号`thread_n_end`。

第62行~第67行根据分块大小分配空间并初始化。

第70行~第84行根据线程 ID 对应的行分块编号`thread_id_m`和列分块编号`thread_id_n`，将对应分块的数据从原始矩阵`A`、`B`、`C`拷贝到临时矩阵`A_padding`、`B_padding`、`C_padding`。

第87行~第95行调用练习2的`gemm_kernel_opt_avx`内核计算分块矩阵乘法，并将计算结果保存回原始矩阵`C`。

1. 按子任务要求，参考`src/lab3/openmp_gemm_baseline.cpp`，利用OpenMP库设计基于多核的支持更大规模的矩阵乘法计算的算法，并将逻辑填入`src/lab3/openmp_gemm_opt.cpp`指定位置（第7行）

```
6  void openmp_gemm_opt(int thread_num, float *C, float *A, float *B, uint64_t M, uint64_t N, uint64_t K){
7      // TODO: 练习3的性能优化任务
8  }
```

2. 验证新算法的正确性

```
1  # build目录下执行
2  cmake -B . -S ../ && cmake --build ./ --target lab3_gemm_opt_openmp.unittest
3  ./dist/bins/lab3_gemm_opt_openmp.unittest
```

Loading [Contrib]/a11y/accessibility-menu.js

```
lenovo@10:~/test1/TA/try/computer_archtecture_labs/build$ ./dist/bins/lab3_gemm_
opt_openmp.unittest
Running main() from /home/lenovo/test1/TA/try/computer_archtecture_labs/build/_d
eps/googletest-src/googletest/src/gtest_main.cc
[==========] Running 4 tests from 1 test suite.
[----------] Global test environment set-up.
[----------] 4 tests from openmp_gemm_opt
[ RUN      ] openmp_gemm_opt.test0
[       OK ] openmp_gemm_opt.test0 (0 ms)
[ RUN      ] openmp_gemm_opt.test1
[       OK ] openmp_gemm_opt.test1 (52 ms)
[ RUN      ] openmp_gemm_opt.test2
[       OK ] openmp_gemm_opt.test2 (432 ms)
[ RUN      ] openmp_gemm_opt.test3
[       OK ] openmp_gemm_opt.test3 (3809 ms)
[----------] 4 tests from openmp_gemm_opt (4294 ms total)

[----------] Global test environment tear-down
[==========] 4 tests from 1 test suite ran. (4294 ms total)
[  PASSED  ] 4 tests.
```

3. 对比优化后的算法与基线的性能（测试用例根据自己的优化选择），根据算法和结果分析优化方案的优缺点

```
# build目录下执行
cmake -B . -S ../ && cmake --build ./ --target lab3_gemm_opt_openmp
./dist/bins/lab3_gemm_opt_openmp 8 512 512 512
```

```
lenovo@10:~/test1/TA/try/computer_archtecture_labs/build$ ./dist/bins/lab3_gemm_
opt_openmp 8 512 512 512
--- Performance before openmp strategy optimization ---
GEMM performance info:
                M, K, N: 512, 512, 512
                Ops: 0.268435
                Total compute time(s): 0.709941
                Cost(s): 0.0035497
                Benchmark(Gflops): 75.6219
--- Performance for after openmp strategy optimization ---
GEMM performance info:
                M, K, N: 512, 512, 512
                Ops: 0.268435
                Total compute time(s): 0.710553
                Cost(s): 0.00355276
                Benchmark(Gflops): 75.5568
---------------------------
Performance difference(Gflops): -0.0651332
```

实验结果说明

以上截图中的性能对比双方均为未优化的 baseline 方案，此处仅作为实验过程的示例和参考。同学们进行此步骤时应使用步骤1实现的优化后的 `openmp_gemm_opt.cpp` 与 baseline 进行对比。

Loading [Contrib]/a11y/accessibility-menu.js